YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND MANAGEMENT
(A CONSTITUENT UNIT OF YENEPOYA (DEEMED TO BE UNIVERSITY))
BALMATTA, MANGALORE

LOW LEVEL DESIGN
ON
SUPPLY CHAIN MANAGEMENT SYSTEM

SUBMITTED BY
ABHAY V

REGNO :
23BCAICD004
LH-05

GUIDED BY
MS. YUKTHA

SUBMISSION DATE
24-04-2026

Table of Contents

1. Introduction
   1.1 Purpose
   1.2 Scope
   1.3 System Overview
2. Module-Level Design
   2.1 Inventory Module
   2.2 Shipment Module
   2.3 Validation Module
3. Detailed Function Design
   3.1 Inventory Functions
   3.2 Shipment Functions
   3.3 Helper Functions
4. Database / Data Structure Design
   4.1 Inventory Data Structure
   4.2 Shipment Data Structure
   4.3 In-Memory Store Operations
5. API Endpoint Specifications
   5.1 Inventory Endpoints
   5.2 Shipment Endpoints
6. Error Handling Design
   6.1 Validation Errors
   6.2 Business Logic Errors
   6.3 Server Errors
7. Class, Object Design and Algorithms
   7.1 Pydantic Models
   7.2 Response Schemas
   7.3 ID Generation Algorithm
   7.4 Record Search Logic
   7.5 Status Transition Logic
8. Conclusion
9. References

1. Introduction

1.1 Purpose

This Low-Level Design (LLD) document provides a detailed technical specification of the internal implementation of the Supply Chain Management System. While the HLD describes the overall architecture, this document specifies the functions, data structures, algorithms, API contracts, class definitions, and error handling strategies that constitute the actual implementation — serving as a direct guide for developers.

1.2 Scope

This document covers:

- Internal structure of Inventory and Shipment modules including all functions and logic.
- Pydantic model definitions for request bodies and response schemas.
- In-memory data structure design and all operations performed on them.
- Detailed API endpoint specifications including HTTP methods, parameters, and response formats.
- Error handling design and algorithms used for ID generation, record search, and status transitions.

1.3 System Overview

The Supply Chain Management System is a FastAPI application exposing seven REST API endpoints across two domains: Inventory and Shipments. It uses Python in-memory lists as the data store, Pydantic for validation, and FastAPI routing for HTTP handling. All models are defined using Pydantic BaseModel classes to ensure type safety and automatic validation throughout the request lifecycle.

2. Module-Level Design

2.1 Inventory Module

The Inventory Module manages all inventory-related data and operations. It maintains a Python list as the in-memory store and exposes functions for all CRUD operations. Module file: inventory.py

- Responsibilities: Create, retrieve, update, and delete inventory records.
- Internal State: inventory_store = [] — Python list holding inventory dictionaries.
- Dependencies: Pydantic InventoryItem model, FastAPI router, helper functions.
- Exposed Routes: GET /inventory, POST /inventory, PUT /inventory/{id}, DELETE /inventory/{id}.

2.2 Shipment Module

The Shipment Module manages all shipment-related data and operations. It maintains a separate in-memory list and references inventory items by ID. Module file: shipments.py

- Responsibilities: Create, retrieve, and update shipment records. Validate that referenced inventory items exist.
- Internal State: shipment_store = [] — Python list holding shipment dictionaries.
- Dependencies: Pydantic ShipmentItem model, FastAPI router, inventory module helper functions.
- Exposed Routes: POST /shipments, GET /shipments, PUT /shipments/{id}.

2.3 Validation Module

Validation is implemented through Pydantic BaseModel classes defined alongside each module. FastAPI automatically triggers validation on all incoming request bodies before any route handler is called.

- InventoryCreate: Validates name (str), quantity (int, >= 0), location (str).
- ShipmentCreate: Validates item_id (int), destination (str), quantity (int, > 0).
- ShipmentUpdate: Validates status (str) — must be one of: Pending, In Transit, Delivered.

3. Detailed Function Design

3.1 Inventory Functions

get_all_inventory() — Returns the entire inventory_store list. Return: List of inventory dicts.

create_inventory_item(item: InventoryCreate) — Generates a unique ID, builds a dict from Pydantic model fields, appends to inventory_store, and returns the new record including the generated ID.

update_inventory_item(item_id: int, item: InventoryCreate) — Searches inventory_store for a record matching item_id. If found, updates fields in-place and returns the updated record. If not found, raises HTTP 404.

delete_inventory_item(item_id: int) — Searches inventory_store for a matching record. If found, removes it by rebuilding the list without the matched entry. Raises HTTP 404 if not found. Returns a confirmation message.

3.2 Shipment Functions

get_all_shipments() — Returns the entire shipment_store list as-is.

create_shipment(shipment: ShipmentCreate) — Validates the referenced item_id exists in inventory_store. If valid, generates a unique shipment ID, sets status to 'Pending', constructs and appends the shipment dictionary. Raises HTTP 404 if item_id is not found.

update_shipment(shipment_id: int, update: ShipmentUpdate) — Searches shipment_store for the matching ID. If found, updates the status field with the validated new value and returns the updated record. Raises HTTP 404 if not found.

3.3 Helper Functions

find_by_id(store: list, record_id: int) — Iterates through the store and returns the first dict where 'id' matches record_id. Returns None if no match found.

generate_id(store: list) -> int — Returns max(item['id'] for item in store) + 1 if store is non-empty, else returns 1. Guarantees unique IDs within the session.

4. Database / Data Structure Design

4.1 Inventory Data Structure

Each inventory record is a Python dictionary stored in inventory_store = []:

```
{ "id": <int>, "name": <str>, "quantity": <int>, "location": <str> }
```

Lookups are performed by linear scan. There is no index structure in this version.

4.2 Shipment Data Structure

Each shipment record is a Python dictionary stored in shipment_store = []:

```
{ "id": <int>, "item_id": <int>, "destination": <str>,
  "quantity": <int>, "status": <str> }
```

Initial status is always 'Pending'. Status transitions are validated against the allowed values list before updating.

4.3 In-Memory Store Operations

- INSERT: record = build_dict(item); store.append(record)
- SELECT ALL: return store
- SELECT BY ID: next((r for r in store if r['id'] == id), None)
- UPDATE: for r in store: if r['id'] == id: r.update(fields)
- DELETE: store[:] = [r for r in store if r['id'] != id]

5. API Endpoint Specifications

5.1 Inventory Endpoints

- GET /inventory — 200 OK — Returns array of all inventory objects.
- POST /inventory — 201 Created — Body: { name, quantity, location }. Returns created item with ID. 422 on validation failure.
- PUT /inventory/{id} — 200 OK — Body: { name, quantity, location }. Returns updated item. 404 if not found.
- DELETE /inventory/{id} — 200 OK — Returns { message: 'Item deleted successfully' }. 404 if not found.

5.2 Shipment Endpoints

- POST /shipments — 201 Created — Body: { item_id, destination, quantity }. Returns shipment with status='Pending'. 404 if item_id not found.
- GET /shipments — 200 OK — Returns array of all shipment objects.
- PUT /shipments/{id} — 200 OK — Body: { status }. Must be: Pending | In Transit | Delivered. Returns updated shipment. 404 if not found.

6. Error Handling Design

6.1 Validation Errors

Validation errors occur when the request body does not conform to the Pydantic model schema. FastAPI automatically handles these and returns:

- HTTP Status: 422 Unprocessable Entity.
- Response Body: JSON object with a 'detail' array containing field-level error descriptions (field name, error type, human-readable message).
- Trigger Conditions: Missing required fields, incorrect data types, constraint violations (e.g., negative quantity).
- Handling: Fully automatic via Pydantic and FastAPI — no custom code required.

6.2 Business Logic Errors

Business logic errors occur when input is structurally valid but violates domain rules:

- Record Not Found — HTTP 404: Raised when PUT or DELETE references an ID not in the store. Response: { "detail": "Item not found" }.
- Invalid Reference — HTTP 404: Raised when POST /shipments receives an item_id not matching any inventory record. Response: { "detail": "Inventory item not found" }.
- Invalid Status — HTTP 422: Raised when PUT /shipments/{id} receives a status value outside the allowed set.

6.3 Server Errors

Unhandled exceptions are caught by FastAPI's global handler:

- HTTP Status: 500 Internal Server Error.
- Response Body: { "detail": "Internal server error" } — no stack traces or implementation details included to prevent information leakage.

7. Class, Object Design and Algorithms

7.1 Pydantic Models

```
class InventoryCreate(BaseModel):
    name: str
    quantity: int = Field(..., ge=0)
    location: str

class ShipmentCreate(BaseModel):
    item_id: int
    destination: str
    quantity: int = Field(..., gt=0)

class ShipmentUpdate(BaseModel):
    status: str  # Validated: Pending | In Transit | Delivered
```

7.2 Response Schemas

InventoryResponse extends InventoryCreate by adding id: int. Returned by POST and PUT /inventory endpoints. ShipmentResponse extends ShipmentCreate by adding id: int and status: str. Returned by POST and PUT /shipments.

7.3 ID Generation Algorithm

```
def generate_id(store: list) -> int:
    if not store: return 1
    return max(item['id'] for item in store) + 1
```

Deleted IDs are never reused within a session to avoid accidental references to removed records.

7.4 Record Search Logic

```
def find_by_id(store: list, record_id: int):
    for record in store:
        if record['id'] == record_id: return record
    return None
```

O(n) linear scan. Acceptable for expected data volumes in this version. Future versions will use indexed database lookups.

7.5 Status Transition Logic

```
ALLOWED_STATUSES = ['Pending', 'In Transit', 'Delivered']
def validate_status(status: str):
    if status not in ALLOWED_STATUSES:
        raise HTTPException(422, f'Invalid status: {status}')
```

No strict sequential state machine is enforced in v1.0 — any valid status may be set regardless of the current state. Future versions will enforce ordered transitions.

8. Conclusion

The Low-Level Design of the Supply Chain Management System provides a comprehensive and precise implementation blueprint covering all modules, functions, data structures, API endpoint contracts, error handling strategies, class definitions, and core algorithms required to build the system.

The Inventory and Shipment modules are cleanly separated with well-defined function signatures and internal state. Pydantic models ensure type-safe validation at every API boundary. In-memory Python lists provide fast, simple data access suitable for the current version, and all data operations are abstracted through helper functions to allow easy migration to a persistent database in future releases.

The error handling design covers validation errors, business logic violations, and unexpected server failures with appropriate HTTP status codes and informative response bodies. The algorithms for ID generation, record lookup, and status validation are simple, deterministic, and well-suited to the scope of this version.

This LLD directly supports the implementation phase of the project and provides a solid reference for developers, testers, and reviewers to understand every design decision made at the code level.

9. References

- FastAPI Official Documentation — https://fastapi.tiangolo.com
- Pydantic v2 Documentation — https://docs.pydantic.dev
- Python Official Documentation — https://docs.python.org/3/
- HTTP Status Codes Reference — MDN Web Docs — https://developer.mozilla.org/en-US/docs/Web/HTTP/Status
- Uvicorn ASGI Server — https://www.uvicorn.org
- RESTful API Design Best Practices — Apigee.
- Project Source Code — Internal repository.